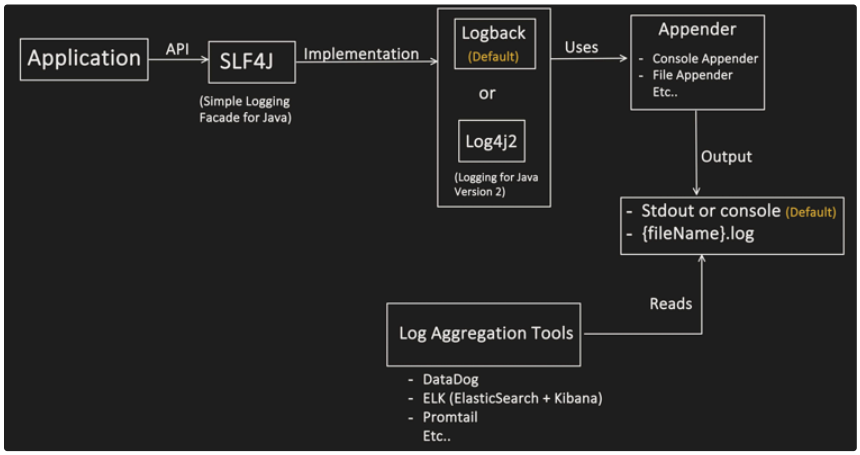
Distributed Logging - Part1

Logging:

It means recording important application events (like requests, errors, warnings, debug info) so that we developers can monitor and debug what the application is doing at runtime.

Before we go with Distributed Logging, lets first see, how its done in Single application:

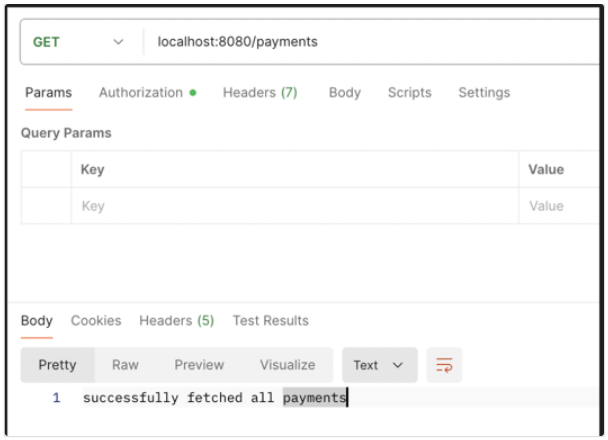


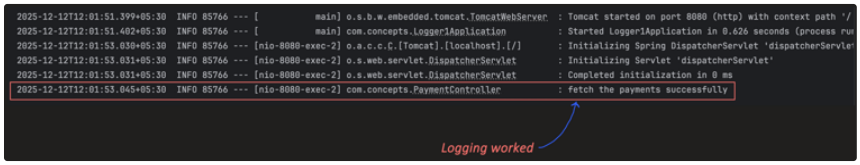
Concept && Coding

Lets see, what it takes to implement the Log functionality:

```
@RestController
public class PaymentController {

    //Get Logger object (Logback or Log4j2)
    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        // Start Logging
        log.info("fetch the payments successfully");

        return "successfully fetched all paym

```

That's it.





But how? What exactly happened?

1. **Dependency**

When we add below dependency

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
</dependency>
```

Concept && Coding

It automatically brings below Jars:

- slf4j-api.jar
- logback-classic.jar
- logback-core.jar

One question might comes to your mind:

Hey Shrayansh, so by default Springboot brings the "*Logback*" library. But what if I want to use "*Log4j2*".

Then I have to add the dependency of "*Log4j2", like below:*

```
<dependency>
        <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

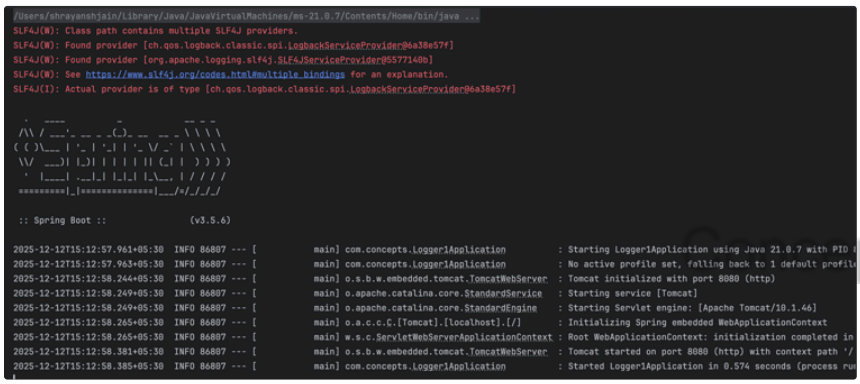
Now, in class path we have 2 implementation library of Slf4j:

- Logback (default) one
- Log4j2, which we have manually added.

Which implementation library will be used now?

```java
/************* LoggerFactory  (Framework code) *******/

private final static void bind() {
try {
              List<SLF4JServiceProvider> providersList = findServiceProviders();
              reportMultipleBindingAmbiguity(providersList);
              if (providersList != null && !providersList.isEmpty()) {

                //From List of providers, it will do get(0)
                PROVIDER = providersList.get(0);
                      .
                      .
                      .
             }
     }

```

Also while starting the application, it will warns us about it:
Note: its just a warning not an error msg, so build will get success.



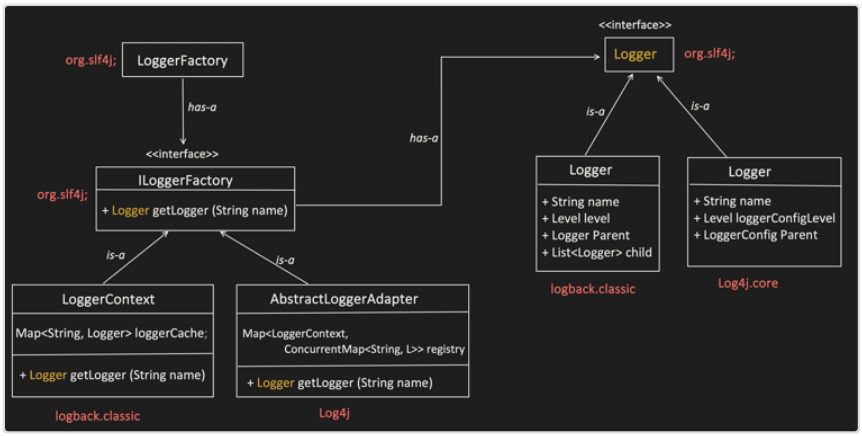
pt && Coding

But for production, get(0) is not reliable, instead we should specifically exclude the default library, if we are manually adding the library.

```pom.txt
/*Excluding the starter logging library, which internally brings:
         - logback-classic
         - logback-core
*/

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>

  //Manually add the implementation library which we want to use.
<dependency>
       <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

2. **Get Logger Object**

Logger log = LoggerFactory.*getLogger*(PaymentController.class);



LoggerFactory does:

- Finds which *slf4j* implementation to use i.e. *Logback* or *Log4j2* etc.
  - and It choose the implementation by seeing which library is present in the class path.

Concept && Coding

- Invokes *LoggerContext* of that specific implementation of slf4j.

LoggerContext does:

- Does caching based on Logger Name. Logger name could be class name or package name or any String.

- Map<String, Logger> loggerCache;

  If for a particular name, Logger object is already present, it re-use it else creates new and put into the cache.

Logger Object:

Few important fields present within Logger class are:

- String name
- Level level
- Logger Parent
- List<Logger> child

name: This is the name of the Logger. It can be :

- Package name

- Class name
- Or any String

For a given logger name, only 1 logger object is created and reused.

Note: In real world application, the common practice is to create 1 logger per class.

```
// logger name = com.concepts.PaymentController
Logger log = LoggerFactory.getLogger(PaymentController.class);

```

level:

- For each Logger object, we configure a log level.

There are multiple level:

- And each Log statement implicitly has its own level based on the method used.

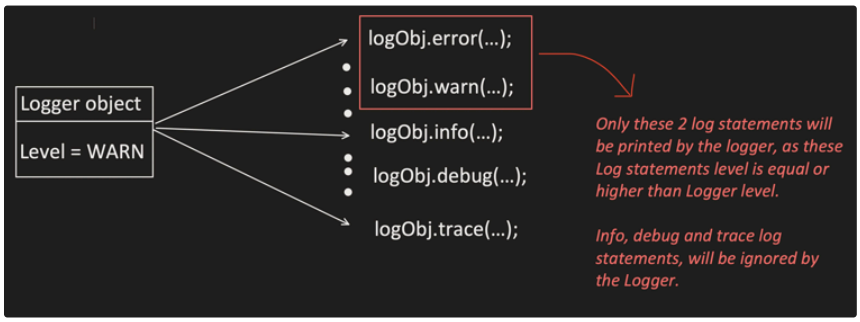
Concept && Coding

  log.debug("...");

  log.info("...");

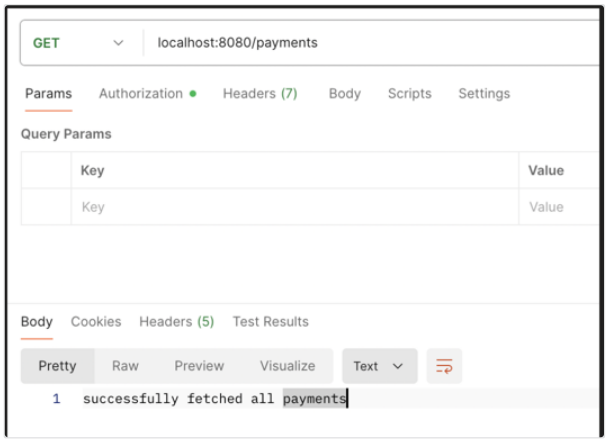
  log.error("...");

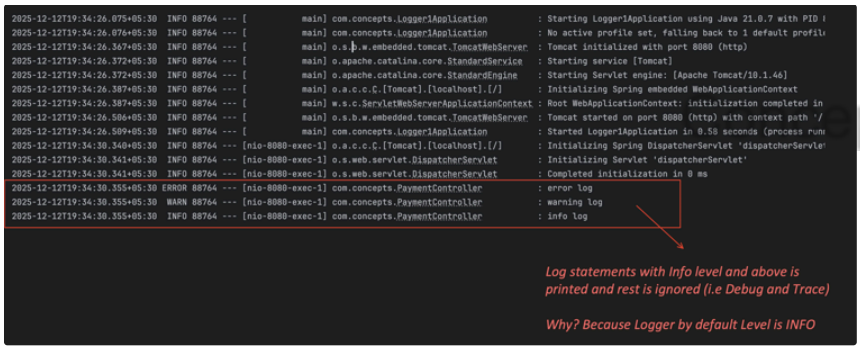
- and Logger prints a logs statement only if log statement level is same or higher than the Logger configured level.




```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {
```

```
8
9           log.error("error log");
10          log.warn("warning log");
11          log.info("info log");
12          log.debug("debug log");
13          log.trace("trace log");
14
15          return "successfully fetched all payments";
16      }
17  }
18
```





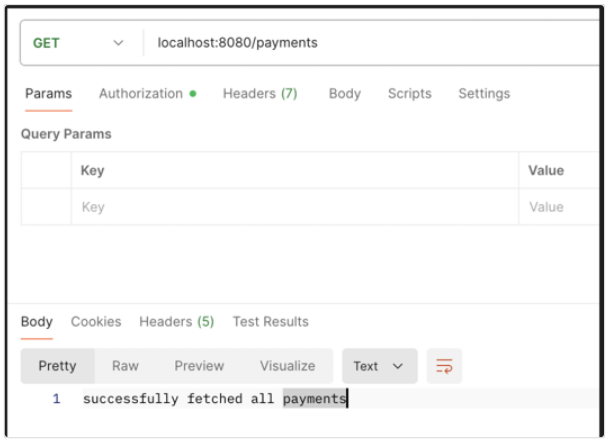
pt && Coding

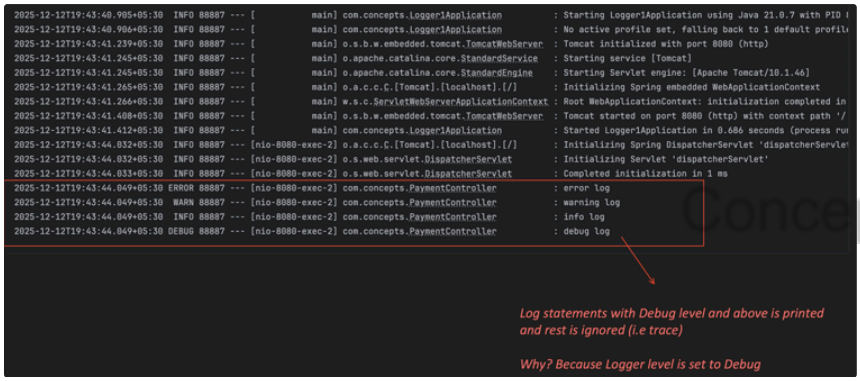
Now, lets change the Logger level through configuration:

```
1  @RestController
2  public class PaymentController {
3
4      Logger log = LoggerFactory.getLogger(PaymentController.class);
5
6      @GetMapping("/payments")
7      public String getPayments() {
8
9          log.error("error log");
10         log.warn("warning log");
11         log.info("info log");
12         log.debug("debug log");
13         log.trace("trace log");
14
15         return "successfully fetched all payments";
16     }
17 }
18
```

application.properties

```
logging.level.com.concepts.PaymentController=DEBUG
```



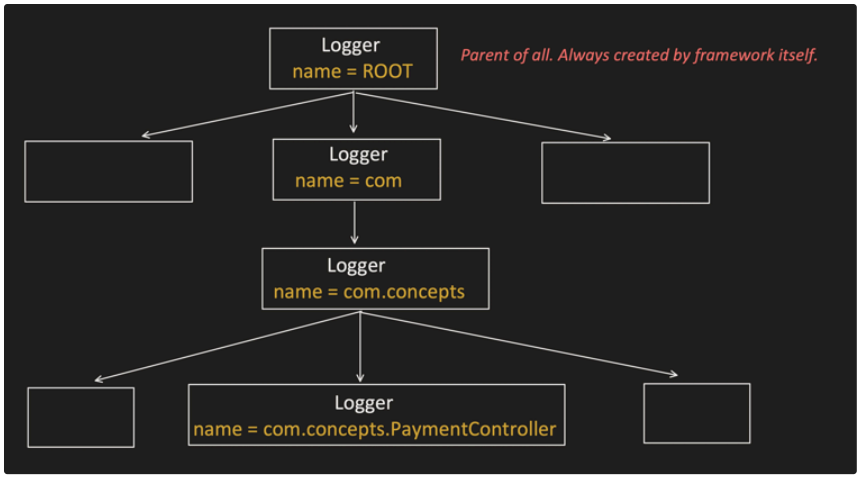


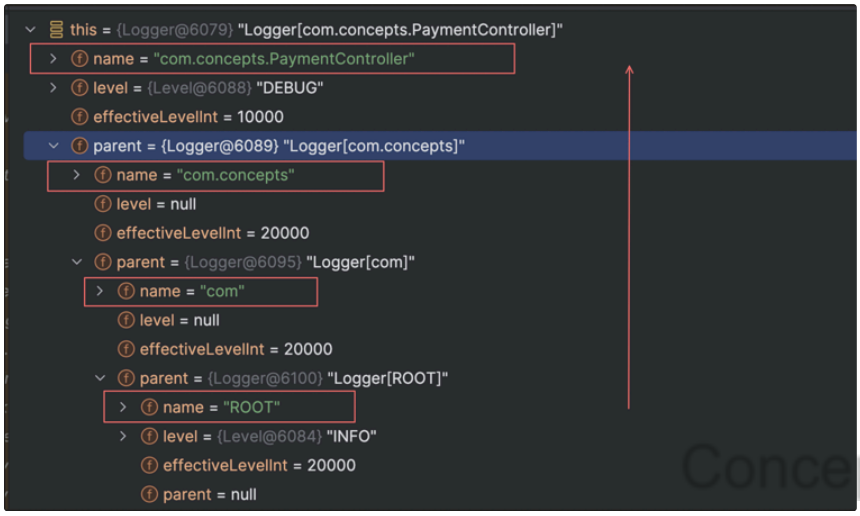
Concept && Coding

- **Logger parent**
- **List<Logger> child:**

Logger framework internally maintains hierarchy of Logger objects

```
//Logger name = com.concepts.PaymentController
Logger log = LoggerFactory.getLogger(PaymentController.class);
```





Concept && Coding

Advantage of maintaining the Parent, Child relationship are:

1. **Log level inheritance:**

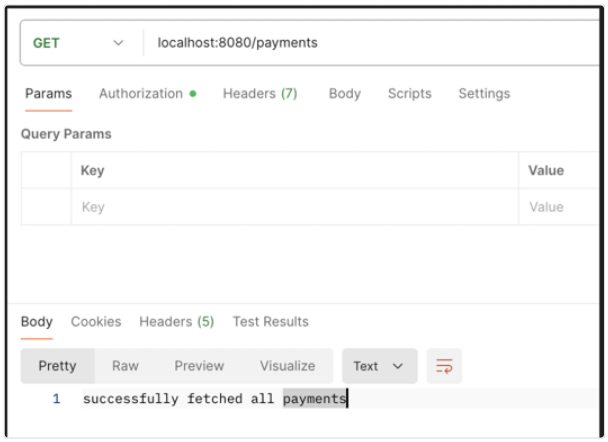
We don't need to configure each and every class.

For ex: I can set level on a Package logger and all of its child automatically inherit it.

```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.error("error log");
        log.warn("warning log");
        log.info("info log");
        log.debug("debug log");
        log.trace("trace log");

        return "successfully fetched all payments";
    }
}

```

application.properties

```

/*This logger level is inherited by all its child loggers
unless those child loggers explicitly override the level.
*/

logging.level.com=DEBUG

```



Concept && Coding

2. **Override specific child behavior**

```
logging.level.com=DEBUG

//I have set a different log level for a specific child logger under "com".
logging.level.com.concepts.PaymentController=WARN

```

3. **Accepted logs are propagated upwards:**

If log is rejected : STOP (do not propagate it)

```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.trace("trace log");

```

```
        return "successfully fetched all payments";
    }
}

```

```

logging.level.com.concepts.PaymentController=DEBUG

```

for ex: *log.trace("trace log statement")*

This log is rejected, as Logger level is DEBUG. So Debug and Upper level is only entertained rested is skipped.

Only accepted log is propagated upwards (till ROOT) and all appenders are executed.

By default is TRUE, but we can turn it FALSE also.

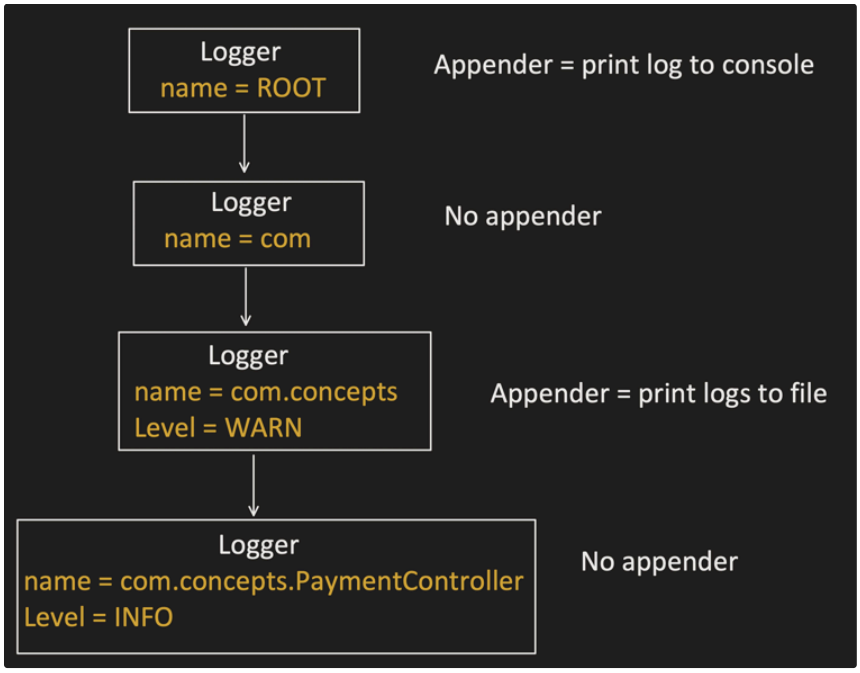
<u>What is appender:</u>

Appender is the component that decides where the log will go:

- console
- file
- DB
- kafka

Etc.

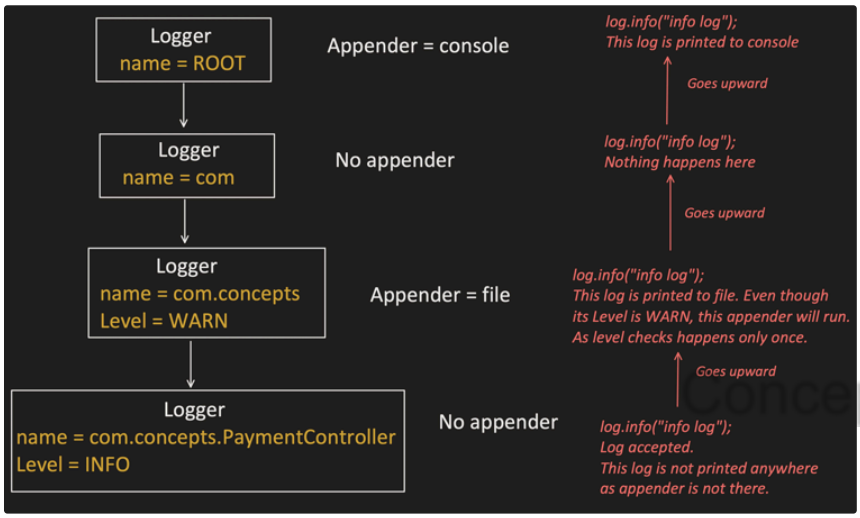
Concept && Coding




```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

```

```
    @GetMapping("/payments")
    public String getPayments() {

      //This log statement is accepted.As PaymentController Logger level is INFO

        log.info("info log");

        return "successfully fetched all payments";
    }
}

```

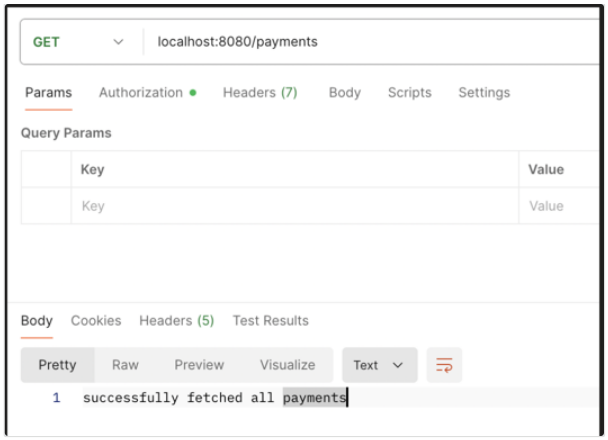
Now, Once a child logger accepts a log event, it **ALWAYS** propagates upward, and all parent appenders execute, regardless of parent levels.

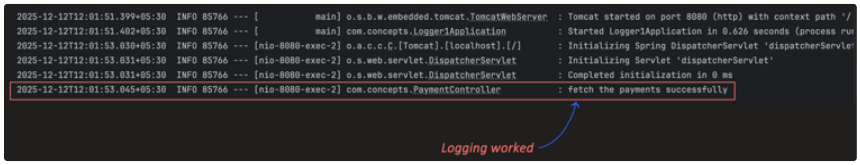


And if you notice once thing, in all our previous example, we have not written any appenders and still we able to see the longs on our **CONSOLE.** How?

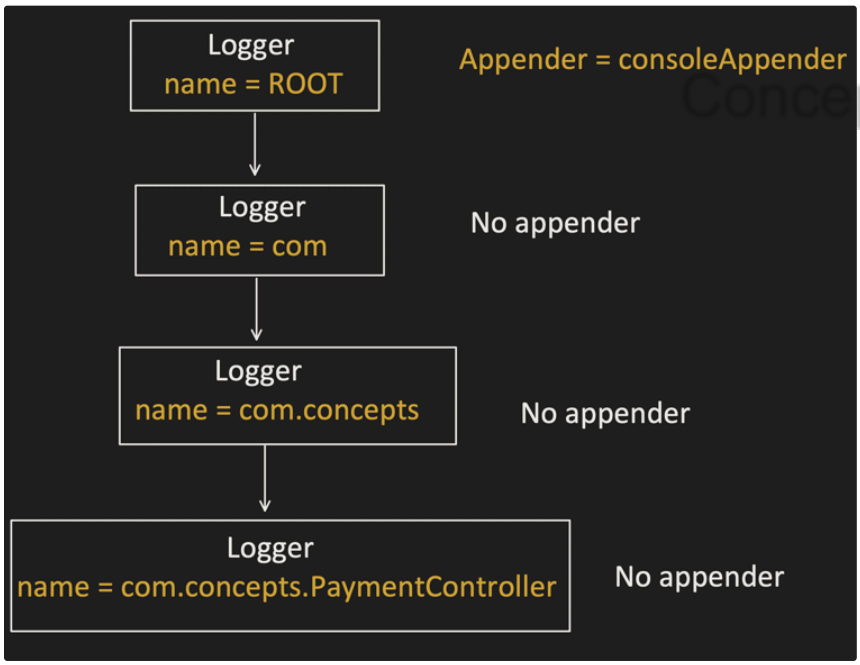
As we know, appender is required to determine where the logs will go (console, file, DB etc..)
So how come its goes to CONSOLE, who is deciding it?

```
@RestController
public class PaymentController {

    //Get Logger object (Logback or Log4j2)
    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        // Start Logging
        log.info("fetch the payments successfully");

        return "successfully fetched all paym

```





So, when we don't specify the appender, accepted Log propagate upwards and at ROOT logger (which framework provides) it has CONSOLE appender present, which print the logs on CONSOLE.



Concept && Coding